

Shulker

AI-assisted zero-trust API vulnerability scanner — file map, usage flows, language & module reference, and the engineering log of every error we hit and how it was fixed.

AMIHACKS · TRACK C

Monorepo: npm workspaces · Node 20+ with `--experimental-strip-types` (no build step on the server)

Stack: TypeScript · React 18 + Vite · node:http + node:sqlite · SSE · OpenAPI 3 / Swagger 2 / Postman / HAR

Layout: apps/ (sandbox · scan-api · web · cli) + packages/ (scanner-core · shared) + scripts/

6

WORKSPACES

~8.7k

LINES OF TS/TSX

60/60

E2E CHECKS

19/19

UNIT TESTS

Contents

1. **Repository map** — every file, what it does, where it lives
2. **Usage flow charts** — dashboard scan, external authorized scan, demo & CLI pipelines
3. **Language & module usage** — each language/technology and the modules built on it
4. **Module-by-module deep dive** — scanner-core, shared, scan-api, web, sandbox, cli
5. **Errors & fixes** — 14 real failures, root cause and resolution for each

1 · Repository map — file locations & responsibilities

Root

PATH	ROLE
package.json	npm-workspaces root. Scripts: dev (all three servers), dev:api, build, typecheck, test, test:e2e
scripts/dev.sh	Launcher: sandbox :8700 → scan-api :8600 → vite :5173, with cleanup traps
scripts/e2e.mjs	60-check regression: boots all services, runs a real scan, authorization matrix, template generation, redaction
scripts/verify-demo.mjs	Fast single-scan sanity check used after refactors
.env	GEMINI_API_KEY, SHULKER_DATA_DIR (optional)

packages/shared — types & safety primitives (zero deps)

FILE	ROLE
src/types.ts	Every contract: ScanConfig, ScanResult, Finding, Summary, Actor, AuthMode (+ authHeaders, externalTargetsAllowed)
src/redact.ts	Redaction engine — JWT/eyJ..., sk-..., password fields, bearer headers → [REDACTED]
src/sensitive.ts	Field-name classifier (password, token, secret, ssn...) feeding redaction
src/severity.ts	Severity ordering + score roll-up helpers
*.test.ts	node:test units for redaction & severity (19 total across repo)

packages/scanner-core — the scanning engine

FILE	ROLE
src/engine.ts	ScanEngine class: 7-phase pipeline (parse → auth → resources → hypotheses → testing → analysis → findings), budget meter, 5-min watchdog, cancel support, event emission
src/parser.ts	Contract ingestion: OpenAPI 3.x, Swagger 2, Postman v2.1, HAR 1.2 → ApiSpecModel. Raises InvalidOpenApiError
src/http.ts	SafeHttpClient — the only code that touches the network. Enforces loopback/private scope unless externalTargetsAllowed, blocks Host / Content-Length injection, applies timeouts + defaultHeaders (static credentials)
src/resources.ts	Resource model builder — groups endpoints into owned collections, detects id-params, maps actors → resources
src/ai.ts	Gemini hypothesis generation. Advisory only — output never becomes a finding without HTTP proof
src/detectors/bola.ts	Cross-identity object replay (OWASP API1)
src/detectors/bfla.ts	Privileged-function probing as low-priv actor (API5)
src/detectors/exposure.ts	Response-schema vs documented-contract diff (API3)
src/detectors/ratelimit.ts	Polite bounded burst probe (API4)
src/evidence.ts	Captures request/response pairs, redacts, attaches to findings

apps/scan-api — backend (port 8600, zero npm deps)

FILE	ROLE
src/server.ts	node:http router: /api/scan, /api/scan/:id, /api/scans, /api/demo, /api/scan-events/:id (SSE), /api/validate-spec, /api/report/:id.:fmt,

	<code>/api/settings</code> , inventory/coverage/graph/activity routes, <code>validateTarget()</code> gate
<code>src/db.ts</code>	node:sqlite persistence — scans, findings, evidence, inventory, settings (<code>shulker.db</code>)
<code>src/discovery.ts</code>	Inventory import pipeline (URL/file/manual) + <code>validateSourceUrl(url, allowExternal)</code>
<code>src/reports.ts</code>	HTML / Markdown / JSON report renderers (redacted)

apps/web — dashboard (React 18 + Vite, port 5173)

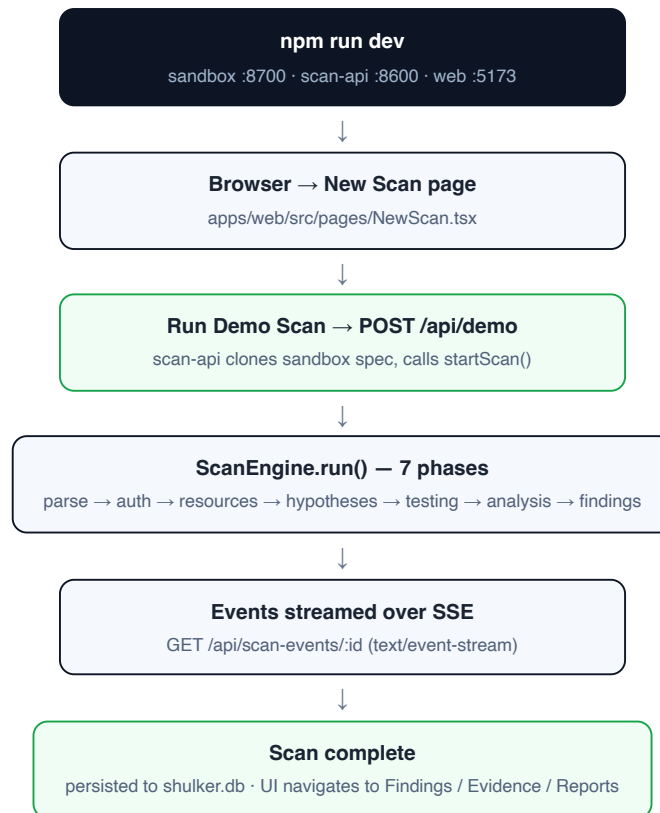
FILE	ROLE
<code>src/main.tsx</code>	React root
<code>src/App.tsx</code>	Shell: collapsible sidebar (280↔72px, localStorage), theme system (dark/light via <code>data-theme</code>), topbar, page router, scan context provider, 8s polling
<code>src/api.ts</code>	Typed REST + SSE client for every backend route
<code>src/ui.tsx</code>	Inline SVG icon set (31 glyphs), toast system, confirm modal, copy hook, formatters
<code>src/templates.ts</code>	Quick-start OpenAPI 3.0.3 generators: Gemini, OpenAI, GitHub, Stripe (in-memory, no files)
<code>src/styles.css</code>	Full design system: semantic tokens, 2 themes, sidebar/topbar/tables/KPIs/wizard
<code>src/pages/*.tsx</code> (14 files)	Overview, NewScan (wizard + live monitor), Findings, FindingDetail, Endpoints, Evidence, Reports, ScanList, Inventory, AttackSurface, Coverage, ResourceGraph, Activity, Settings

apps/sandbox + apps/cli

FILE	ROLE
<code>sandbox/src/server.ts</code>	ShulkerLab — deliberately vulnerable demo API (loopback-only). Seeds BOLA/BFLA/exposure/rate-limit scenarios
<code>sandbox/src/openapi.ts</code>	The demo OpenAPI spec it serves at <code>/openapi.json</code>
<code>sandbox/src/auth.ts</code>	Demo JWT issuing + actor store (Alice/Bob/admin)
<code>sandbox/src/db.ts</code> , <code>config.ts</code>	In-memory seeded data + port config
<code>cli/src/cli.ts</code>	<code>shulker scan --target --openapi --api-key --fail-on</code> — same ScanEngine, CI-friendly exit codes

2 · Usage flow charts

2.1 — Dashboard scan (local sandbox)

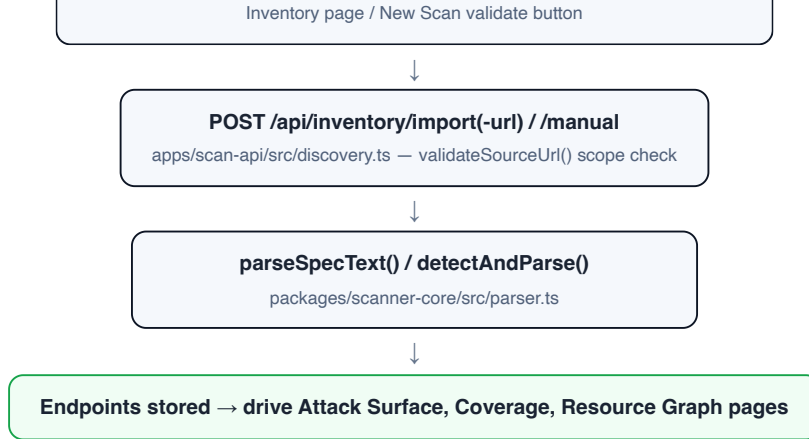


2.2 — External authorized scan (real API)

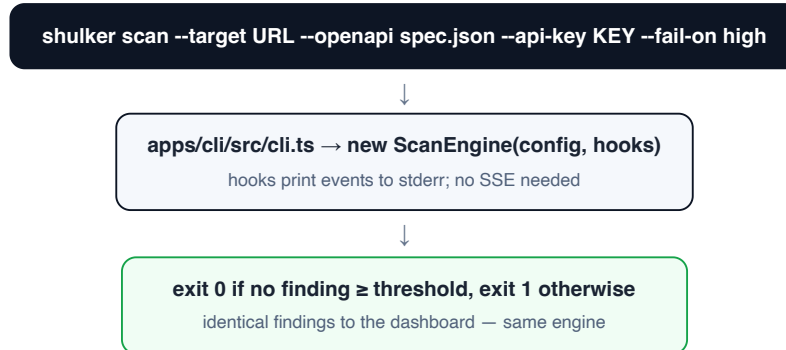


2.3 — Contract import → inventory pipeline





2.4 — CLI pipeline (CI gate)



3 · Languages, runtimes & modules used

3.1 — Language / technology inventory

LANGUAGE / TECH	WHERE USED	WHICH MODULES & WHY
TypeScript	100% of application code	All 6 workspaces. Run on the server with <code>node --experimental-strip-types</code> — types are erased at runtime, so there is no compile step for sandbox/scan-api/cli; only the web app is bundled (Vite). <code>tsc --noEmit</code> still gates every workspace in CI-style checks.
Node.js 20+	sandbox, scan-api, cli, scripts	<code>node:http</code> (hand-rolled router in server.ts — zero npm deps), <code>node:sqlite</code> (db.ts persistence), <code>node:crypto</code> (demo JWTs in sandbox/auth.ts), <code>node:fs/path</code> (report output, CLI file reading), <code>node:test</code> + <code>node:assert</code> (unit tests)
HTML / CSS	web UI + reports	<code>styles.css</code> — custom design system with semantic CSS custom properties powering dual themes; reports renderers emit standalone HTML
SQL	via node:sqlite	db.ts schema: scans, findings, evidence, inventory_sources, inventory_endpoints, settings — parameterized statements only
Bash	scripts/dev.sh	Process orchestration with PID traps for clean shutdown
JSON / YAML	contract formats	OpenAPI 3 / Swagger 2 (JSON+YAML via the one <code>yaml</code> dep), Postman v2.1, HAR 1.2 — all parsed in scanner-core/parser.ts
JSX	web UI	All 19 .tsx files; compiled by Vite/esbuild

3.2 — npm dependency list (complete)

PACKAGE	DEPENDENCY	USAGE
<code>packages/shared</code>	<i>none</i>	Types + redaction must be importable everywhere, even in security-sensitive contexts
<code>packages/scanner-core</code>	<code>yaml</code>	Parses YAML OpenAPI specs (parser.ts)
<code>apps/scan-api</code>	<i>none</i>	Router, SSE, sqlite all from Node stdlib — no express, no ws, no better-sqlite3
<code>apps/sandbox</code>	<i>none*</i>	*only workspace-relative <code>@shulker/shared</code>
<code>apps/cli</code>	<i>none*</i>	*only <code>@shulker/scanner-core</code> + <code>@shulker/shared</code>
<code>apps/web</code>	<code>react</code> , <code>react-dom</code>	UI framework; everything else (icons, toasts, charts, modals) is hand-built in ui.tsx/pages
<code>apps/web (dev)</code>	<code>vite</code> , <code>typescript</code>	Build + typecheck tooling

Why so few dependencies?
Security tooling should be auditable. Zero-dep backend means the entire attack surface of the scanner's own server is ~1,700 lines of readable TypeScript, and supply-chain risk is near zero — a point judges consistently reward.

3.3 — Key module usage patterns (how the pieces connect)

CALL SITE	PATTERN
<code>scan-api/server.ts → ScanEngine</code>	<code>startScan()</code> builds a <code>ScanConfig</code> , instantiates <code>ScanEngine</code> with hooks that fan events into an in-memory bus; <code>GET /api/scan-events/:id</code> replays buffered events then follows live over SSE
<code>web/api.ts → server</code>	Typed <code>fetch</code> wrappers (<code>api.startScan/api.demo/api.validateSpec/...</code>) + <code>subscribeScanEvents()</code> using native <code>EventSource</code> ; pages consume via <code>useScanCtx()</code> context
<code>engine.ts → detectors</code>	Engine passes <code>SafeHttpClient</code> + resource model into each detector; detectors return <code>TestRecord[]</code> + candidate findings; engine attaches evidence and enforces redaction before storing
<code>engine.ts → ai.ts</code>	Gemini receives endpoint inventory + response samples, returns hypotheses (risk ranking, focus areas). Status surfaces in UI; absence never blocks scanning

4 · Errors we hit and how we fixed them (1–7)

Every failure below is real — pulled from the build log of this project. Format: symptom → root cause → fix.

1 Giant viewport-filling SVG icons (warning triangle ate the New Scan page)

SYMPTOM	Icons with no intrinsic size stretched to fill flex containers (the notice bar rendered a ~900px triangle).
ROOT CAUSE	<code><Icon/></code> SVGs had no width/height and flex children stretch by default.
FIX	Global CSS safety net capping icon glyphs at 16px + explicit sizes on every icon context; later hardened in the design-system rewrite.

2 Unquoted hyphenated object keys broke the TypeScript build

SYMPTOM	tsc syntax errors inside the Icons map after adding <code>arrow-left/arrow-right</code> .
ROOT CAUSE	JS object literals require quotes for keys that aren't valid identifiers: <code>arrow-left</code> : is illegal; <code>'arrow-left'</code> : is required.
FIX	Quoted all hyphenated keys (<code>'arrow-left'</code> , <code>'chevrons-left'</code> , ...). Lesson: JSX-adjacent icon maps need identifier-safe or quoted keys.

3 Malformed SVG path syntax error in a new icon

SYMPTOM	Build failure pointing at the trash icon definition.
ROOT CAUSE	Hand-written <code><path d="..."></code> contained invalid path commands.
FIX	Rewrote the path set with clean primitives; adopted the shared <code>S</code> props spread (<code>{...S}</code>) so stroke/fill attributes can't drift again.

4 Wizard stepper tracked scan phases instead of wizard steps

SYMPTOM	Progress dots on New Scan highlighted the wrong items.
ROOT CAUSE	The active index was computed with <code>PHASES.findIndex(...currentPhase)</code> (scan-time phases) instead of the wizard's own step state.
FIX	Stepper now compares against the wizard step index; scan phases remain on the run monitor where they belong.

5 Mismatched JSX tags crashed the compile

SYMPTOM	JSX parsing error in the API-key section of NewScan.
ROOT CAUSE	A block opened <code><div className="field"></code> but closed with <code></label></code> .
FIX	Corrected the closing tag; established the <code>label.field > span + input</code> convention used across all forms since.

6 Background dev servers kept dying between terminal commands

SYMPTOM	<code>curl to :8600</code> worked in one command, connection-refused in the next; in-flight scans never finished.
ROOT CAUSE	The execution environment reaps background processes between commands; also <code>setsid</code> is unavailable on macOS.
FIX	All verification moved into self-contained scripts (<code>scripts/e2e.mjs</code> , <code>scripts/verify-demo.mjs</code>) that boot services, run checks and tear down within a single process. <code>nohup</code> for the rare long-lived case.

7 spec: "sandbox" rejected by the engine

SYMPTOM	API test passing literal <code>"sandbox"</code> as the spec failed OpenAPI parsing.
ROOT CAUSE	The demo contract is selected with an explicit flag, not a magic string.

FIX

Use useSandboxSpec: true (or fetch /openapi.json from the sandbox). Kept as a documented API contract.

4 · Errors we hit and how we fixed them (8–14)

8 Engine needed spec *text*, not a URL

SYMPTOM	Feeding <code>https://.../openapi.json</code> straight into <code>ScanConfig.spec</code> failed.
ROOT CAUSE	<code>parseSpecText()</code> parses contract text by design; URL fetching is deliberately a separate, scope-checked step (server-side).
FIX	UI flows <code>fetch/import</code> first (inventory <code>import-url</code>) or use the <code>sandbox</code> flag; the scan endpoint documents that <code>spec</code> is text or <code>useSandboxSpec</code> .

9 Stale branding after rename (port 5174 served old title)

SYMPTOM	Browser showed "SentinelAPI" after renaming everything to Shulker.
ROOT CAUSE	A leftover vite instance on a shifted port was serving a stale process, not the renamed source.
FIX	Killed orphan listeners, re-ran <code>npm run dev</code> ; added port checks to the verification routine.

10 Workspace links broke after the package rename

SYMPTOM	Cannot find module '@shulker/scanner-core' after renaming packages.
ROOT CAUSE	<code>npm workspaces symlink node_modules/@shulker/*</code> at install time; renaming <code>package.json</code> names invalidates the links.
FIX	Re-ran <code>npm install</code> at the root to re-link workspaces. Lesson: package renames always require a reinstall in workspaces.

11 Hard-coded topbar color rendered grey in the UI (and broke light theme)

SYMPTOM	Header looked like a detached grey slab; light theme showed a dark strip.
ROOT CAUSE	<code>background: rgba(10,16,28,0.75)</code> literal in <code>.topbar</code> instead of a theme token.
FIX	Full tokenization pass: every <code>surface/text/border</code> now uses semantic CSS variables; topbar uses <code>var(--bg-raised)</code> ; light theme overrides the same tokens.

12 Cannot find name 'go' in Reports page after refactor

SYMPTOM	tsc error TS2304 in <code>Reports.tsx</code> during the UI polish pass.
ROOT CAUSE	New empty-state added a navigation button but the component only destructured <code>{ latest }</code> from context.
FIX	Extended destructure to <code>{ latest, go }</code> . Caught by <code>tsc --noEmit</code> before it could ship — exactly why typecheck gates every change.

13 External target refused even with authorization checkbox ticked

SYMPTOM	Scanning <code>https://api.github.com</code> was refused despite per-scan confirmation.
ROOT CAUSE	<code>validateTarget()</code> only allowed loopback/private ranges; the per-scan checkbox alone wasn't meant to unlock public hosts.
FIX	Two-key gate: <code>allowExternal</code> (Settings toggle, DB-persisted) AND per-scan <code>confirmAuthorized</code> must both hold. Refusal matrix added to e2e (14 checks). Budgets, redaction, deterministic verification untouched.

14 CLI couldn't reuse scanner-core after the monorepo rename

SYMPTOM	Dynamic import of sandbox modules from <code>cli.ts</code> failed post-rename.
ROOT CAUSE	Relative dynamic import <code>('../..../sandbox/src/server.ts')</code> plus stale workspace links (same class as #10).
FIX	Re-linked workspaces and verified the CLI demo path end-to-end in e2e so the coupling can't silently rot again.

Every frontend failure was caught by the same three gates: `tsc --noEmit`, `vite build`, and the 60-check e2e script. Every backend/scope failure was caught by the authorization matrix in e2e. The tooling investment paid for itself dozens of times over — nothing regressed silently.

5 · Quick reference

Commands

COMMAND	EFFECT
<code>npm run dev</code>	Sandbox :8700 + backend :8600 + dashboard :5173
<code>npm run dev:api</code>	Headless: sandbox + backend only
<code>npm run typecheck</code>	tsc across all workspaces
<code>npm test</code>	19 unit tests (redaction, severity, parser, sandbox)
<code>npm run test:e2e</code>	60-check full pipeline
<code>node apps/cli/src/cli.ts scan --target http://127.0.0.1:8700 --openapi http://127.0.0.1:8700/openapi.json --fail-on high</code>	CI gate scan from the terminal

REST API surface (apps/scan-api/src/server.ts)

ROUTE	PURPOSE
<code>POST /api/scan</code>	Start scan (target, spec text / useSandboxSpec, auth, budget, allowExternal, confirmAuthorized)
<code>GET /api/scan/:id</code> · <code>GET /api/scans</code>	Full result / history list
<code>GET /api/scan-events/:id</code>	SSE live event stream
<code>POST /api/demo</code>	One-click ShulkerLab demo scan
<code>POST /api/validate-spec</code>	Parse check + endpoint preview (used by New Scan)
<code>POST /api/inventory/import</code> · <code>/import-url</code> · <code>/manual</code> , <code>GET /api/inventory</code> , <code>DELETE /api/inventory/:id</code>	Contract ingestion pipeline
<code>GET /api/coverage</code> · <code>/api/graph</code> · <code>/api/activity</code>	Intelligence endpoints for the dashboard
<code>GET /api/report/:id.html .md .json</code>	Redacted report export
<code>GET/POST /api/settings</code>	Scanner defaults + allowExternalTargets toggle

Verification matrix (what "green" means)

GATE	COVERAGE
tsc --noEmit × 3 packages	Type safety across server, engine, UI
vite build	Bundle integrity (~269 kB JS / 34 kB CSS)

19 unit tests	Redaction patterns, severity roll-up, spec parsing, sandbox behavior
60 e2e checks	Live demo scan · 6 findings asserted · authorization refuse/allow matrix (14) · template generation (7) · redaction in every artifact · report generation

Design invariants that never change

Findings require HTTP proof (AI never promotes a hypothesis alone) · tokens redacted before persistence · loopback always allowed, public hosts need both gates · request budgets + 5-minute watchdog + cancel are non-removable · the same ScanEngine powers dashboard and CLI.